

Lexical Analyzer & Token Counter

Lexical Analyzer & Token Counter

1. Title

Lexical Analyzer & Token Counter

2. Objective

The objective of this project is to develop a lexical analyzer that reads a source-code file and identifies different types of tokens. The program also counts the number of tokens belonging to each category.

The analyzer identifies:

- Keywords
- Identifiers
- Operators
- Constants/Literals
- String Literals
- Separators/Delimiters
- Special Symbols
- Comments

3. Problem Statement

Develop a program using Python that reads source code from an input file and performs lexical analysis. The program should identify and classify tokens into different categories and display the total number of tokens in each category.

Comments should be ignored during token classification.

4. Algorithm

1. Start the program.
2. Read the source code from `input.txt`.
3. Remove single-line and multi-line comments.
4. Scan the source code character by character using regular expressions.
5. Identify keywords.
6. Identify identifiers.
7. Identify operators.
8. Identify constants and literals.
9. Identify string and character literals.

10. Identify separators and delimiters.
11. Identify special symbols.
12. Display each token with its corresponding token type.
13. Count the number of tokens in each category.
14. Display the token counts.
15. Stop the program.

5. Source Code

The main source code is available in:

```
lexical_analyzer.py
```

The program uses Python regular expressions to recognize different token patterns.

Main Token Categories

Token	Description
Keyword	Reserved words such as <code>int</code> , <code>float</code> , <code>if</code>
Identifier	Variable and function names
Operator	Symbols such as <code>+</code> , <code>-</code> , <code>=</code> , <code>></code>
Constant	Numeric values such as <code>50</code> , <code>2.0</code>
String Literal	Text enclosed in double quotes
Separator	Symbols such as <code>(</code> , <code>)</code> , <code>;</code> , <code>,</code>
Special Symbol	Symbols such as <code>#</code> , <code>&</code> , <code>`</code>
Comment	Text beginning with <code>//</code> or enclosed in <code>/* */</code>

6. Sample Input

```
int sum = a + b;
float average = sum / 2.0;

// Calculate average
if (average > 50)
printf("Pass");
```

7. Sample Output

```
TOKEN TYPE
-----
int          Keyword
sum          Identifier
=            Operator
a            Identifier
+            Operator
b            Identifier
;            Separator
float        Keyword
average      Identifier
```

=	Operator
sum	Identifier
/	Operator
2.0	Constant
;	Separator
if	Keyword
(	Separator
average	Identifier
>	Operator
50	Constant
)	Separator
printf	Identifier
(	Separator
"Pass"	String Literal
)	Separator
;	Separator

TOKEN COUNT

Keywords	: 3
Identifiers	: 7
Operators	: 4
Constants	: 2
String Literals	: 1
Separators	: 8

8. Token Classification

For the given input:

Token Type Count

Keywords	3
Identifiers	7
Operators	4
Constants	2
String Literals	1
Separators	8

Example Classification

Token Type

int	Keyword
sum	Identifier
=	Operator
a	Identifier
+	Operator
2.0	Constant
if	Keyword
(	Separator

Token	Type
"Pass"	String Literal

9. Test Cases

Test Case 1 – Basic Arithmetic

Input:

```
int a = 10;  
int b = 20;  
int c = a + b;
```

Expected result:

- `int` → Keyword
- `a, b, c` → Identifiers
- `10, 20` → Constants
- `=, +` → Operators
- `;` → Separators

Test Case 2 – Conditional Statement

Input:

```
if (a > 10)  
printf("Greater");
```

Expected result:

- `if` → Keyword
- `a, printf` → Identifiers
- `>` → Operator
- `10` → Constant
- `"Greater"` → String Literal
- `(, ), ;` → Separators

Test Case 3 – Comments

Input:

```
// This is a comment  
int x = 25;
```

Expected result:

The comment is ignored and the remaining tokens are classified.

Test Case 4 – Decimal Constant

Input:

```
float average = 25.5;
```

Expected result:

- `float` → Keyword
- `average` → Identifier
- `=` → Operator
- `25.5` → Constant
- `;` → Separator

10. Conclusion

The Lexical Analyzer & Token Counter successfully reads source code from an input file and classifies it into different token categories such as keywords, identifiers, operators, constants, string literals and separators. The program also counts the number of tokens in each category. This project demonstrates the basic working principle of the lexical analysis phase of a compiler.